

Scanned area

Polygon

ASTANA - CANOPY AGGREGATE

51 348 trees

across 14 scans - 72 snapshots

58 3668 13 749

runs avg / scan visible

32% 4.0 m

avg confidence avg crown

FILTERS

COMPUTER SCIENCE • 6B06101

Min confidence 0%

High

RECENT SCANS

Automated Tree Recognition & Green-Space Mapping

We built a **working system** that turns a satellite image of Astana into a tree map — end to end, in a browser. The detection models are the engine inside: the contribution is the whole machine.

AUTHORS Berik Sharipov • Anuar Totin • Rasul Aidarkhanov SUPERVISOR S. Satbayev

AGENDA

What we'll cover

- 01 **Problem & relevance** — why Astana needs this
- 02 **The gap & aim** — off-the-shelf ≈ 0 here
- 03 **What this project really is** — a system, not a model
- 04 **Objectives, problem statement & methods**
- 05 **Literature review** & the gap it leaves
- 06 **The automated pipeline** (architecture)
- 07 **Beyond the minimum** — what we chose to build
- 08 **Data, database & hardware**
- 09 **How we measure success** (the maths)
- 10 **Four detection engines** — YOLO · Mask R-CNN · DeepForest · SAM 2
- 11 **Why several models** & ensembles
- 12 **Results** on one common test set
- 13 **Canopy** — the system in use
- 14 **Discussion & conclusions**

RELEVANCE

Mapping a city's trees — today it's done by hand

Astana is expanding fast. Urban green cover matters for **heat mitigation, irrigation planning and green-space policy** — but to know where the trees are, someone has to **count them one by one** off imagery. That doesn't scale to a whole city, and it's outdated before it's finished.

The real problem isn't "can we beat a benchmark" — it's that **there is no automated process** for this in Astana at all. So the task we set ourselves was an **engineering** one: build that process.

TODAY — MANUAL

Counting trees by eye, image by image. Slow, not repeatable, never finished for a growing city.

OUR GOAL — A SYSTEM

One repeatable tool: point at an area → it finds the trees and puts them on a map, with coordinates and exports. No manual counting.

 WHY WE COULDN'T JUST USE AN EXISTING TOOL

0.012

Box mAP@50

A popular ready-made detector (NEON DeepForest), run as-is on Astana — essentially blind. There was **no off-the-shelf system** that worked here, so building one was the only option.

AIM

Design, build and evaluate an **automated end-to-end system** that recognises trees and maps green space from Astana satellite imagery — delivered as a working tool, with the detection accuracy reported honestly.

THE THESIS, IN ONE IDEA

The deliverable is a **system** — not a model

Our title says "**automated ... system.**" So the unit of work isn't "which neural net scores highest" — it's the whole machine that takes a raw satellite capture and returns a usable, georeferenced tree inventory. Everything else is a **component of building that**:

■ COMPONENT

The dataset

What the engine learns from — a means, not the goal.

■ COMPONENT

The models

Four interchangeable engines that plug into the pipeline.

■ COMPONENT

The experiments

How we tuned the engine — evidence we went deep, not the product.

■ COMPONENT

The UI & features

What makes it usable by a non-technical operator.

We could have shipped the bare minimum — one model, a segmentation mask, no metrics. Instead we built a **tool someone at Zelenstroy could actually operate**. The next slides show the system first; the models come later, as parts of it.

 OBJECTIVES

Five tasks — all in service of the system

- 1 Build an **annotated Astana dataset** (tiled satellite imagery, instance polygons).
- 2 Train and compare four **detection engines** — YOLOv8-seg, Mask R-CNN, DeepForest, SAM 2.
- 3 **Combine** them through ensembles and test whether that helps.
- 4 Evaluate everything on **one common test set** for a fair comparison.
- 5 Wrap it all in an **automated web application** — the deliverable that ties tasks 1–4 together.

METHODS

Supervised deep learning · transfer learning from pretrained weights · sliding-window tiling of large captures · pixel→WGS-84 geo-referencing · mAP@50 as the single comparison metric · software engineering (FastAPI · React · SQLite).

— FORMALLY — WHAT GOES IN, WHAT COMES OUT

Input → Output

INPUT

A satellite capture of an area of Astana at zoom 19 (≈ 0.3 m / px) — uploaded, or drawn on a map and auto-fetched from **ESRI or Google** tiles — cut into overlapping **640 px RGB tiles**.

Plus the operator's choices: **model, confidence, geo-referencing mode**.

OUTPUT

For every detected tree: a **bounding box**, an **instance mask**, a **confidence score**, **crown area** and **lat/lng**.

Tiles are stitched, duplicates merged, results shown on a city **map** with per-area counts and **GeoJSON / CSV / HTML** export.

capture → tile → model $f(\cdot)$ → { box, mask, conf, area, lat/lng }_{per tree} → merge & map & export

— LITERATURE REVIEW • 31 PAPERS • 2019–2025

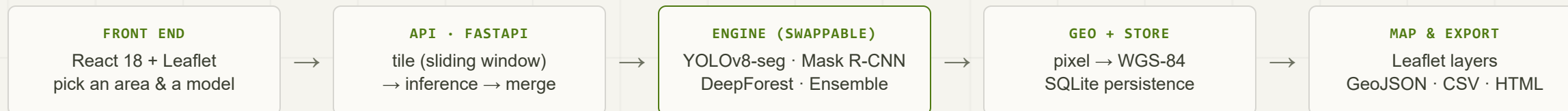
What others report — and the gap each leaves

METHOD	AUTHORS • YEAR	DATA	BEST METRIC	GAP FOR OUR PROBLEM
Mask R-CNN (MCAN)	Lv et al. • 2023	UAV RGB, China	Det AP 92.4%	Forest UAV, not urban satellite; research result only
RetinaNet / Faster R-CNN	dos Santos et al. • 2019	UAV RGB, Brazil	AP 82–93%	Detection alone — no segmentation, no system
YOLOv12m	Abbas & Damaševičius • 2025	RGB satellite	mAP@50 90.8%	Benchmark on a clean public set; no deployable tool
DeepLabV3+ / U-Net	Martins • 2021 / Wang • 2021	Aerial 10–32 cm	F1 91% • IoU 96%	Semantic mask, not per-tree instances; not Central Asia
DeepForest (off→fine)	Ventura et al. • 2024	NAIP 60 cm, USA	F 0.42 → 0.73	US imagery; collapses without local fine-tuning
YOLOv5x	Velasquez-Camacho • 2023	Ground-level, Spain	F1 84.9%	Europe, ground-level; no GIS/coordinate pipeline
DeepForest (urban)	Dakov & Petrova-Antonova • 2024	Aerial, Sofia	F1 0.67–0.69	Closest analogue — but still Europe, still research-only

Every study reports high accuracy – on **its own region and sensor**, as a **research result**. **None was tested on Central Asia, and none delivers a deployable, operator-run system.** That gap – not a higher mAP – is what this project fills.

— THE AUTOMATED PIPELINE — THE CORE OF THE PROJECT

From a satellite image to a tree-map — automatically



ONE AUTOMATED CHAIN

Capture → tile → detect → merge → georeference → store → map → export. **No manual step** between the operator drawing an area and the finished, downloadable inventory.

SWAPPABLE ENGINE — THE KEY DESIGN IDEA

The detection model is just one plug-in step — swap YOLO for Mask R-CNN, DeepForest or an ensemble without touching the rest. A better model tomorrow drops straight in. ~1 km² at zoom 19 in **≈ 18 s** on a laptop GPU.

— BEYOND THE MINIMUM

ENGINEERING EFFORT

A minimal version would do far less — we built a tool

THE BARE MINIMUM

- One model, hard-coded
- Just a segmentation mask
- No coordinates, no export
- No metrics, no comparison
- Run once, nothing saved

WE WENT FURTHER →

WHAT CANOPY ACTUALLY DOES

- + **Two tile providers** — ESRI & Google, fetched in-browser
- + **Model choice** — 7 YOLO sizes · Mask R-CNN · DeepForest ± SAM 2
- + **Ensembles** — Weighted Box Fusion + cross-YOLO vote
- + **Manage scans** — name, hide, delete (cascade)
- + **Live metrics** — tree count, canopy coverage %, crown area
- + **Geo + export** — WGS-84 · GeoJSON / CSV / HTML · persistent map

None of these is required to "detect a tree" — each is a deliberate choice to make the system **usable, comparable and repeatable**. That is where most of the engineering effort went.

— A COMPONENT OF THE SYSTEM — INPUTS & INFRASTRUCTURE

What goes in, and where it runs

DATASET

High-zoom Astana captures (~0.3 m/px). We hand-labelled ~**5 500 tree crowns** across ~**100 images** as instance polygons (~ 8 700 training tiles after 640+128 tiling). A held-out common set of **14 images · 702 trees** scores every model.

DATABASE (SQLITE)



One-to-many, **ON DELETE CASCADE** — delete a scan, its detections go too.

HARDWARE

- RTX 4060 8 GB — YOLO + all common-set inference
- RTX 4070 — Mask R-CNN
- RTX 4050 — DeepForest

8 GB VRAM ceiling → batch 2 + mixed precision. Deliberately modest — to prove it runs without a cluster.



— THE MATHS, IN PLAIN TERMS

Three simple ideas — one comparison number

1 • IOU — DID THE BOX LAND ON THE TREE?

$$\text{IoU} = \frac{\text{overlap area } (A \cap B)}{\text{combined area } (A \cup B)}$$

How much a predicted box overlaps the real one. **1.0** = perfect, **0** = miss. A tree counts as found when **IoU $\geq$ 0.5**.

2 • PRECISION & RECALL

Precision — of the trees we flagged, how many were real.

Recall — of all real trees, how many we found: **ours $\approx$ 30%** at the default setting.

3 • MAP@50 — THE SINGLE SCORE

Sweep the confidence threshold, track precision against recall, and take the **area under that curve** (at IoU $\geq$ 0.5). One honest number to compare every engine the same way — the **0.315** on the results slide.

HOW THE TWO DETECTORS WORK

YOLOv8 — one pass: grid $\rightarrow$ boxes + masks in a single look (fast).

Mask R-CNN — two passes: propose regions, then refine (slower, careful).



— FOUR INTERCHANGEABLE ENGINES · ONE PIPELINE

The system doesn't depend on one model

—

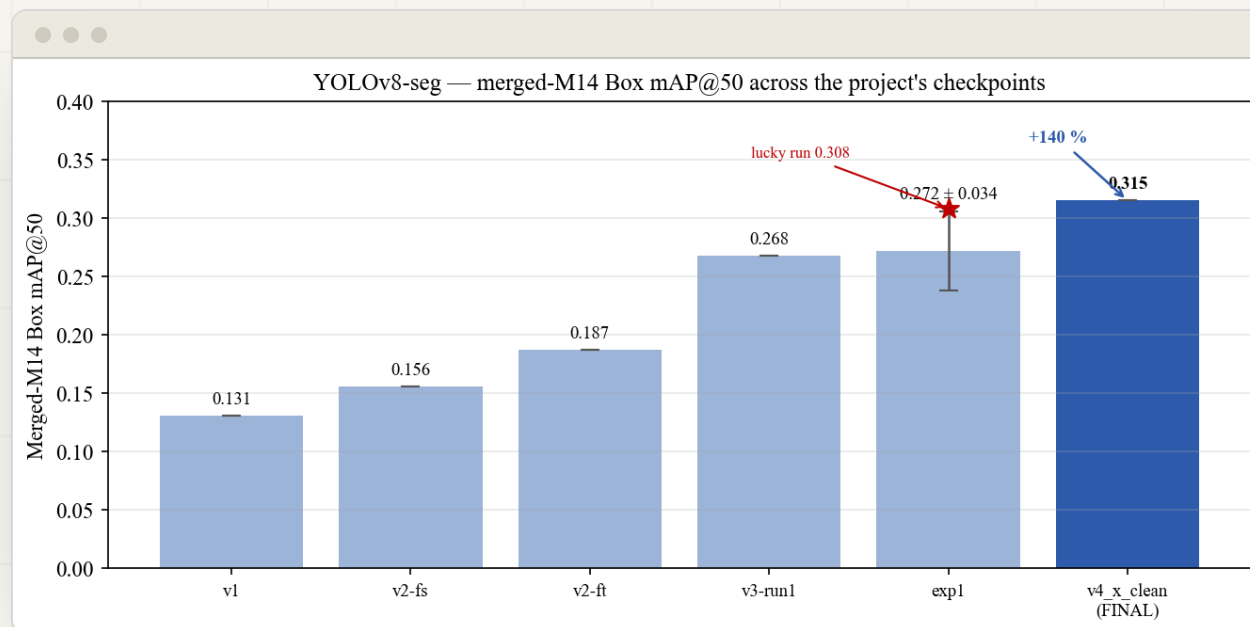
any of four can drive it.

YOLOv8-seg (Rasul) · Mask R-CNN (Berik) · DeepForest + SAM 2 (Anuar) ·
plus two ensembles. Each plugs into the same automated pipeline and is
scored on the same 14 images.

ENGINE #1 • 23 EXPERIMENTS

MY CONTRIBUTION

YOLOv8x-seg: from 0.131 to 0.315



Box mAP@50 across every experiment on the common set.

+140%

Box mAP@50 over my own first model (0.131 → **0.315**).

WHAT ACTUALLY MATTERED

- 1 Start from pretrained (COCO) weights
- 2 Tile at the resolution you deploy at
- 3 Keep augmentation moderate

WHAT I ALSO FOUND

Default augmentation beat hand-tuned (top-down trees are rotation-invariant); model size is **U-shaped**; run-to-run noise $\approx \pm 0.03$ — so small gaps aren't real.

ENGINE #2 • BERIK SHARIPOV

Mask R-CNN — the careful, mask-first engine

A classic **two-stage, region-based** detector. Where YOLO looks once, Mask R-CNN looks twice — propose, then refine — slower, but with naturally **crisp instance masks**. In our project it is one of the swappable engines behind the same automated pipeline.

HOW IT WORKS

A **Region Proposal Network** suggests candidate boxes → **RoIAlign** crops each region → three heads output a **box**, a **class** and a per-pixel **mask**. Segmentation is built in, so outlines hug the crown.

HOW BERIK TRAINED IT

Transfer-learned from a **COCO-pretrained** ResNet-50-FPN Mask R-CNN, fine-tuned on the same Astana tiles (v2 + v3), single **"tree"** class, same split as every engine. 50 epochs, SGD.

HOW IT DOES ON ASTANA

Box mAP@50 **0.166**, Mask **0.158**. At confidence 0.5: **precision 0.44** • **recall 0.22**. Strongest on medium, well-separated crowns; loses the small and shaded ones.

WHY IT STAYS IN THE APP

Below YOLO overall on our data — but it recovers some **larger trees the others miss**, and its masks are the cleanest. That's exactly why the pipeline keeps it as a user-selectable engine.

ENGINE #3 • ANUAR TOTIN

DeepForest + SAM 2 — the forest specialist, re-taught

A two-part engine: **DeepForest** (a RetinaNet-style tree-crown detector) finds the trees, then **SAM 2** turns each box into a high-quality mask. It produced the **0.012** that started the whole project — the clearest proof of why domain adaptation matters.

HOW IT WORKS

DeepForest outputs a box per crown; each box becomes a **prompt** for **SAM 2**, which segments the exact pixels — zero-shot, no extra training.

THE DOMAIN PROBLEM

DeepForest is pretrained on **North-American airborne forest** imagery. Pointed at a dry steppe city it was nearly blind — **0.012**. A mismatch, not a bug: different trees, spacing and sensor.

HOW ANUAR FIXED IT

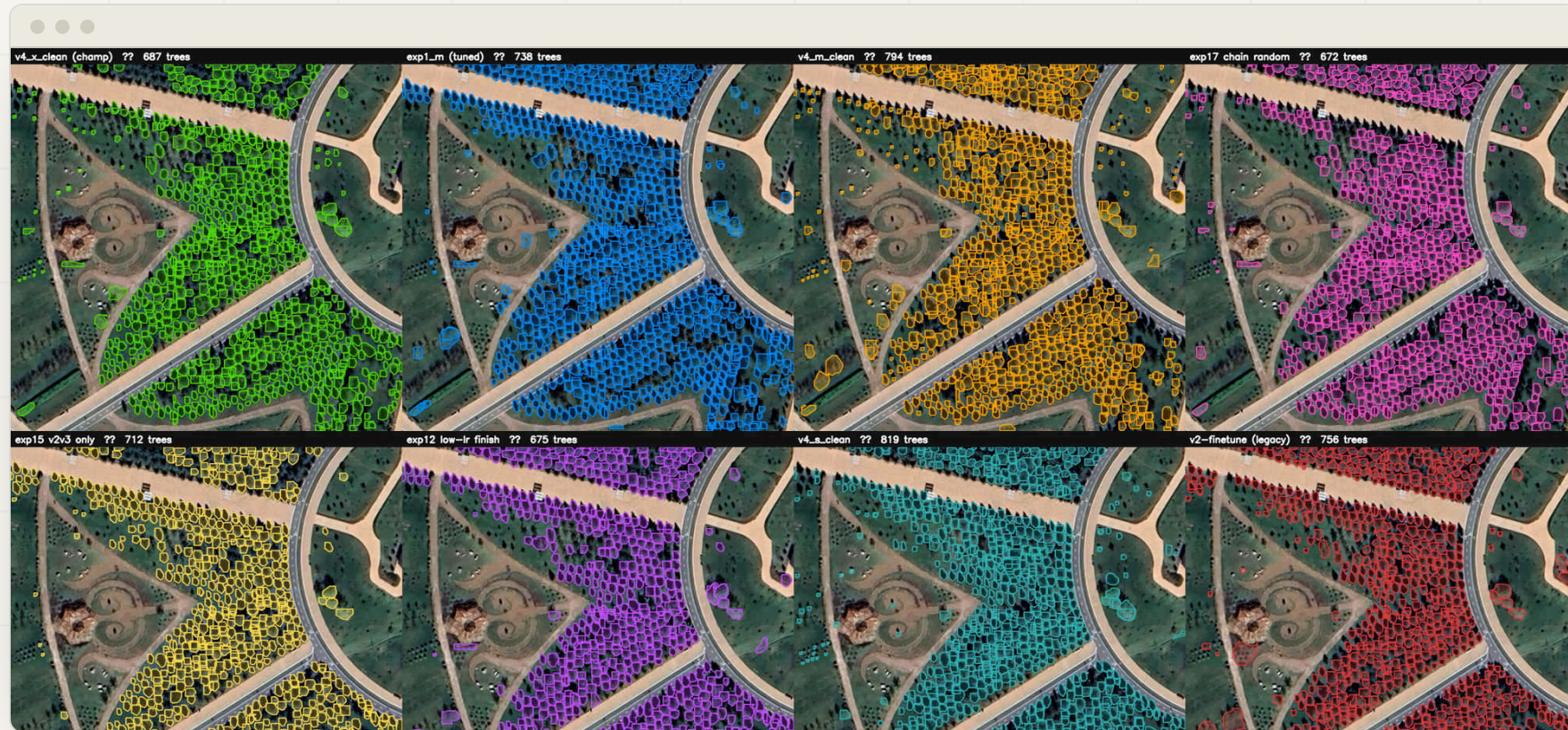
Fine-tuned DeepForest on the Astana data (LR 1e-4, 30 epochs) and paired it with SAM 2 masks (conf 0.3, 400 px patches). Result: **0.012** → **0.146** Box mAP@50 (Mask **0.134**) — a **×12** jump from re-teaching.

WHY IT STAYS IN THE APP

SAM 2 gives the **best general-purpose masks** of any engine, and the off-the-shelf → fine-tuned story is the project's core lesson. Still weak on tiny dense trees — so it's offered alongside the others.

— THE DESIGN RATIONALE — A SYSTEM CHOICE, NOT A MODEL CHOICE

Same garden, eight engines, eight answers



The very same Botanical Garden, scanned by **eight detection engines** — each outlines a **different set of trees** and reports a different count (**672–819**). None is simply "right".

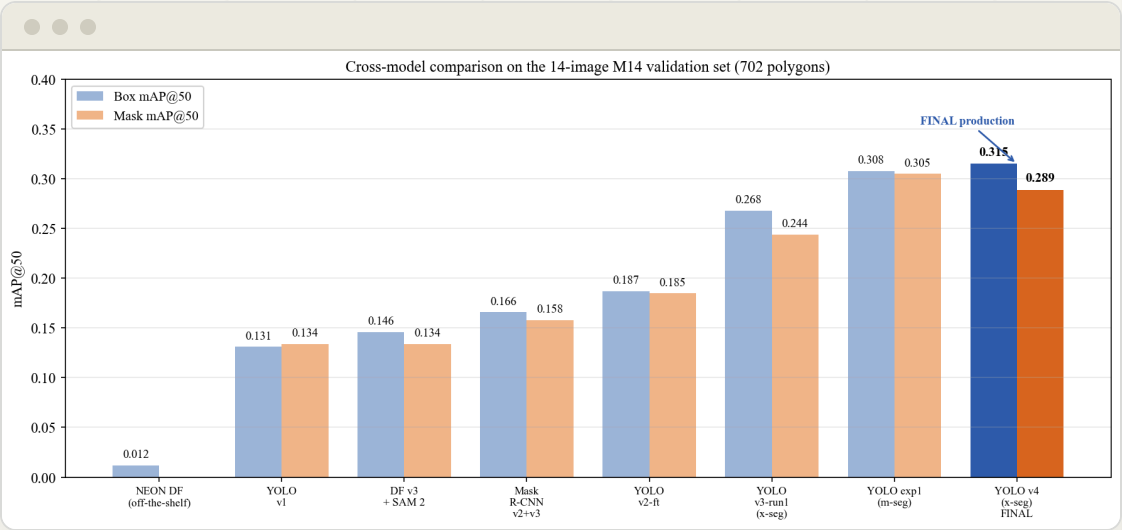
That's why the system lets the operator **switch engines** and combine them — **Weighted Box Fusion** + a **cross-YOLO 4-vote** that drops single-model false positives (e.g. stadium roofs). Choice is a feature.

— HEAD-TO-HEAD • SAME 14 IMAGES • 702 TREES

One common test set, every model

MODEL	BOX MAP@50	MASK MAP@50
YOLOv8x-seg v4 (ours)	0.315	0.289
YOLOv8x-seg v3	0.268	0.244
Mask R-CNN	0.166	0.158
DeepForest + SAM 2	0.146	0.134
YOLO v1 baseline	0.131	0.134
NEON DeepForest (off-the-shelf)	0.012	—

Champion operating point (conf 0.25): **precision 0.52 • recall 0.29**. Margins between top models are small – partly within run-to-run noise.



We read this as "these engines are in the same ballpark on Astana," **not** "one clearly wins." What matters for the project: **any of them can drive the system**.

tree 0.3 tree 0.3 tree 0.5

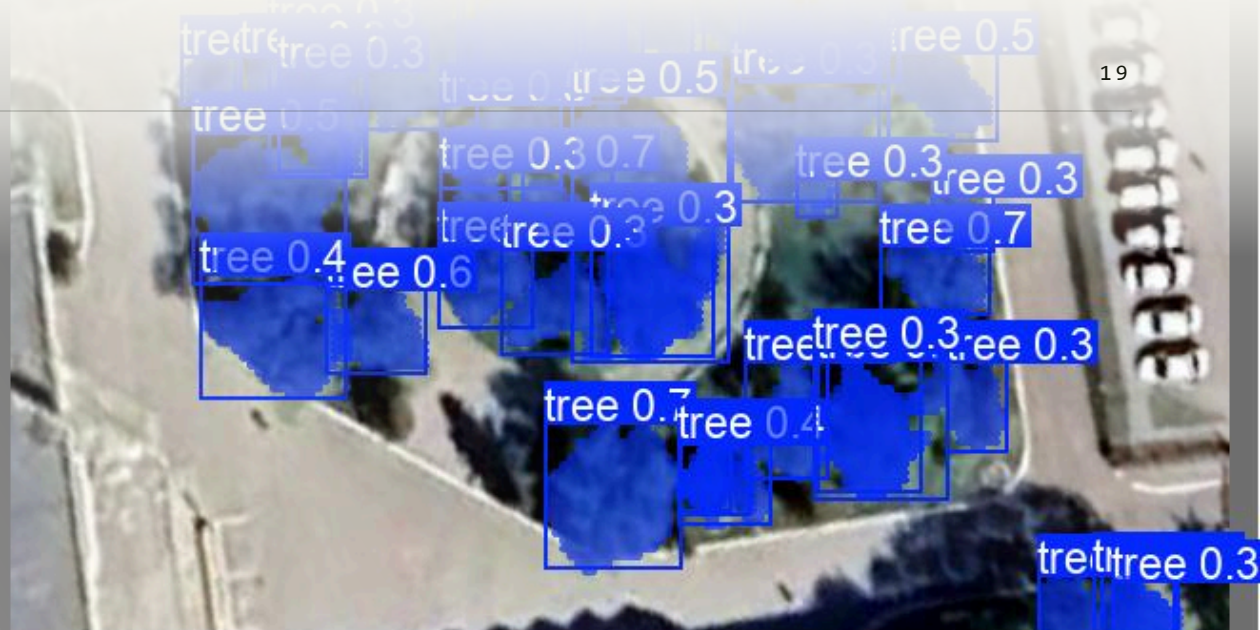
WHAT IT ACTUALLY LOOKS LIKE

Detections vs. real crowns

Predictions line up with real tree crowns in many places. But honestly — it **doesn't find every tree**: at the default confidence it finds about **30%** of labelled trees, and per image it swings from almost none to nearly all.

tree 0.4 tree 0.4 tree 0.6

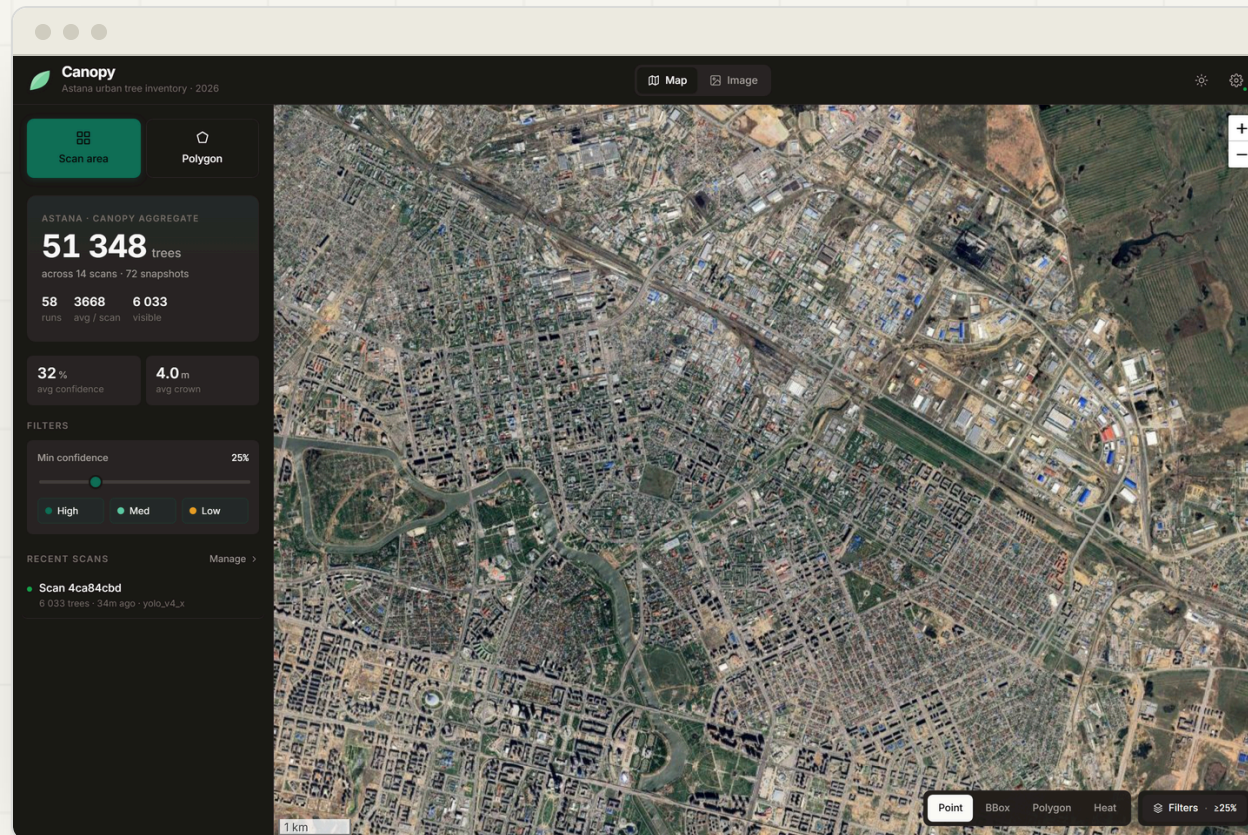
HONEST ABOUT MISSES — SMALL & SHADED TREES



THE DELIVERABLE

MY CONTRIBUTION

Canopy — pick an area, pick a model, scan



ONE FLOW

Select an area of Astana → the app tiles it, runs the chosen model, and draws every detected tree on the map — confidence filter, Point / Box / Polygon / Heat layers.

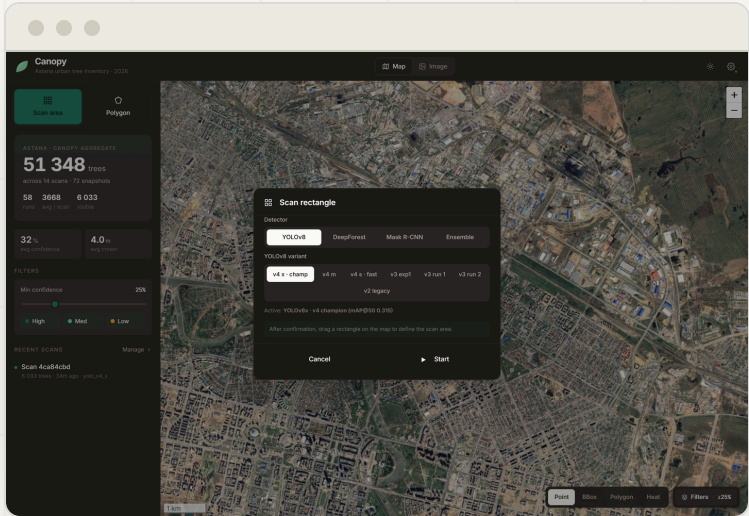
GROWS AS YOU USE IT

Every scan is stored; the city map is the running aggregate of everything ever processed. Coverage and counts update live.

Counts shown in screenshots come from repeated demo scans — **not** a full city census.

THE CHOICES THAT MAKE IT A TOOL, NOT A DEMO

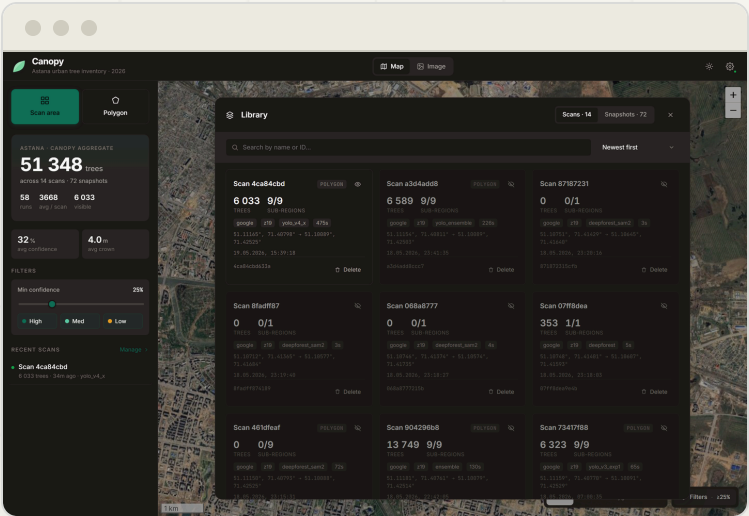
Choose · combine · manage · measure



CHOOSE & COMBINE

Model + ensemble picker

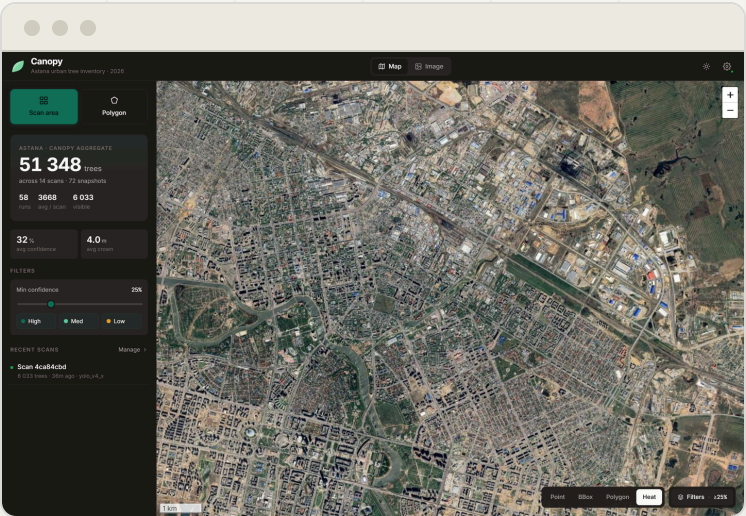
7 YOLO sizes · Mask R-CNN · DeepForest ± SAM 2 · WBF & cross-YOLO vote — chosen per scan, with provider (ESRI / Google) and confidence.



MANAGE

Name · hide · delete

Every scan and snapshot is renamable, hideable and deletable (cascade) from a library modal — so the workspace stays usable over time.



MEASURE

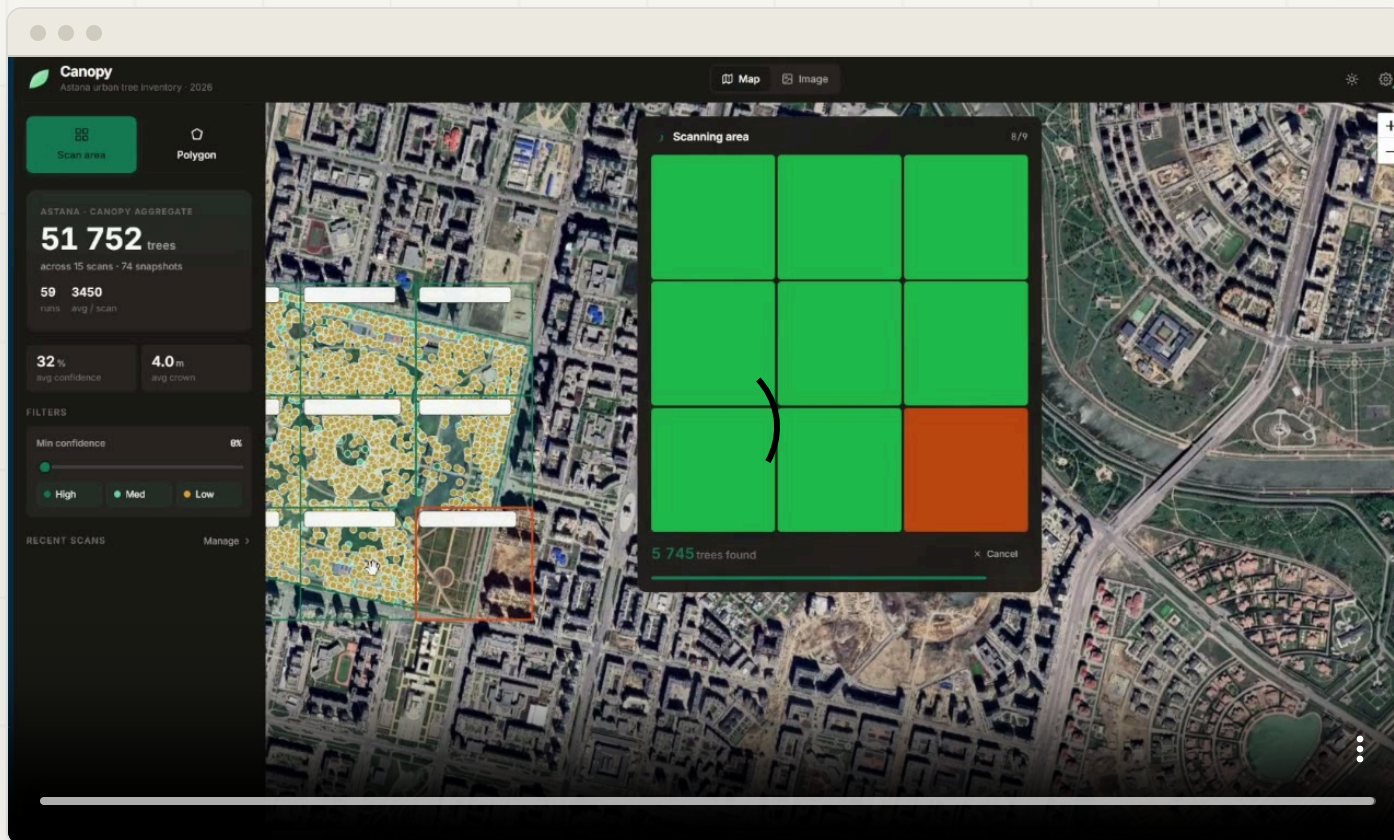
Heat map & coverage

Density heat-map, per-area tree count, **green-space coverage %** and crown area inside a drawn polygon — metrics, not just dots.

— THE PIPELINE RUNNING END-TO-END

MY CONTRIBUTION

Live demo — automated scan of the Botanical Garden



WHAT YOU'RE SEEING

The v4 engine scanning the Botanical Garden end-to-end in Canopy: **tile** → **detect** → **map**, every tree drawn live.

WHY THIS SCENE

A dense, well-defined planting — the clearest case. Honest reminder: on sparser scenes recall drops.

FULL DEMO VIDEO

Plays here automatically (~1 min) — or open the full recording:

https://drive.google.com/file/d/10ceaLC4yMEgLtITHVR8t7_B4VRRPNaf/view

 HONEST ASSESSMENT

What we'd say up front

LIMITATIONS

- Small training set → modest accuracy (0.315 mAP, ≈ 30% recall)
- Domain shift: training captures vs. live map tiles
- No explicit split of heavily-overlapping crowns

BUT ALSO

- Off-the-shelf fails here (0.012) — our models do meaningfully better
- A fair, reproducible comparison on local data
- A working **system**, not just a notebook

None of this is hidden — it's the honest state of a student project, and accuracy improves directly with more annotated data, while the system around it already works.

— OBJECTIVES → OUTCOMES

Every task, addressed

- ✓ **An automated system** — capture → tile → detect → georeference → map → export, with no manual step.
- ✓ **Working application** — Canopy: provider & model choice, ensembles, manage, metrics, export.
- ✓ **Dataset** — annotated, tiled Astana imagery with a held-out 14-image / 702-tree common set.
- ✓ **Four engines + two ensembles**, all scored identically on one set.
- ✓ **Fair evaluation** — the first such measurement for Astana.

The headline isn't a score — it's that a **manual task is now an automated system**. Accuracy is modest (and honestly so: ready-made tools score ~0 here, ours do meaningfully better), but the **system** is the contribution.

Contributions & future work

— MY CONTRIBUTION (RASUL)

- 1 The whole **automated system & Canopy app** (FastAPI + React + SQLite + Leaflet) — the deliverable
- 2 The **YOLOv8 engine** — 23 experiments, +140% over my baseline
- 3 The **cross-YOLO vote** ensemble & the common-set evaluation harness

— FUTURE WORK

- 1 More annotated data → reach the $F1 \approx 0.7$ range
- 2 Train on the **same imagery we run on** (close the domain gap)
- 3 Crown-splitting · species labels · containerised multi-user deploy

ASTANA - CANOPY AGGREGATE

51 348 **THANK YOU**
trees

across 14 scans - 72 snapshots

58 3600 13

runs avg 1.5 min visit

32% 4.0 m

avg confidence

FILTERS

Min confidence 0%

High

RECENT SCANS Manage

Thank you — we'll gladly answer your questions.

From a manual, one-by-one task to an **automated** tree-mapping system for Astana — a working tool anyone can run.

AUTHORS Berik Sharipov · Anuar Totin · **Rasul Aidarkhanov** SUPERVISOR S. Satbayev